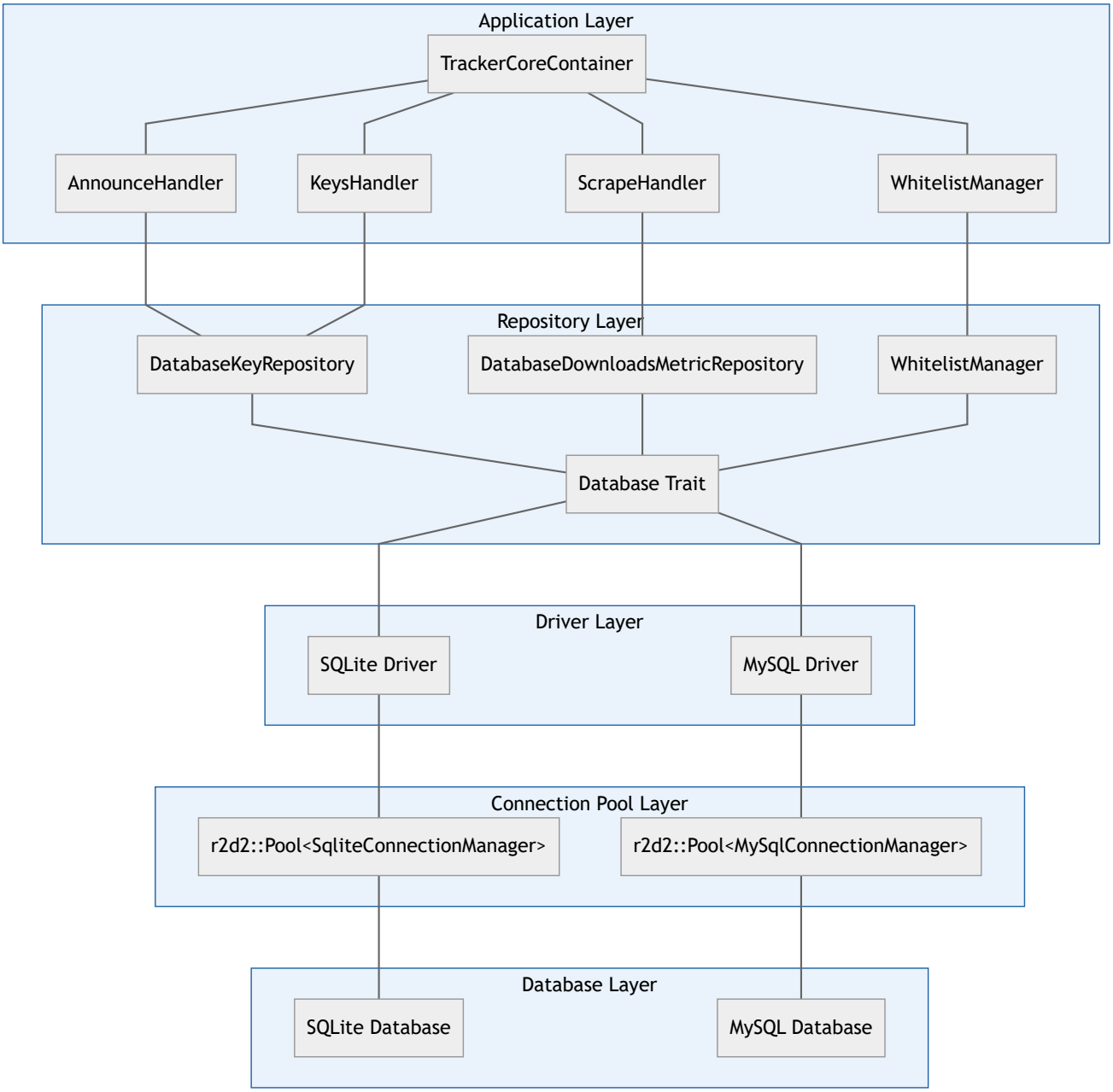
07. 데이터베이스 연동 및 영속성 분석

7.1 데이터베이스 아키텍처 개요

Torrust Tracker는 영속성 계층을 위해 잘 설계된 데이터베이스 추상화 아키텍처를 구현하고 있습니다. 이 아키텍처는 다음과 같은 핵심 특징을 가지고 있습니다:

7.1.1 추상화 계층 구조



7.1.2 핵심 컴포넌트

Database Trait (`packages/tracker-core/src/databases/mod.rs`)

```rust
#[automock]
pub trait Database: Sync + Send {
// 스키마 관리
fn create_database_tables(&self) -> Result<(), Error>;
fn drop_database_tables(&self) -> Result<(), Error>;
// 토렌트 메트릭 관리
fn load_all_torrents_downloads(&self) -> Result<NumberOfDownloadsBTreeMap, Erro
fn save_torrent_downloads(&self, info_hash: &InfoHash, downloaded: u32) -> Resu
fn increase_downloads_for_torrent(&self, info_hash: &InfoHash) -> Result<(), Er
// 화이트리스트 관리
fn load_whitelist(&self) -> Result<Vec<InfoHash>, Error>;
fn add_info_hash_to_whitelist(&self, info_hash: InfoHash) -> Result<usize, Erro
fn remove_info_hash_from_whitelist(&self, info_hash: InfoHash) -> Result<usize,
// 인증 키 관리
fn load_keys(&self) -> Result<Vec<authentication::PeerKey>, Error>;
fn add_key_to_keys(&self, auth_key: &authentication::PeerKey) -> Result<usize,
fn remove_key_from_keys(&self, key: &Key) -> Result<usize, Error>;
}
```

이 트레이트는 모든 데이터베이스 작업을 추상화하여 구체적인 데이터베이스 구현에 독립적인 인터페이스를 제공합니다.

7.2 리포지토리 패턴 구현

7.2.1 리포지토리 패턴의 적용

Torrust Tracker는 도메인 로직과 데이터 접근 로직을 분리하기 위해 리포지토리 패턴을 구현합니다:

DatabaseKeyRepository (`packages/tracker-core/src/authentication/key/repository/persisted.rs`)

```
pub struct DatabaseKeyRepository {
database: Arc<Box<dyn Database>>,
}

impl DatabaseKeyRepository {
pub fn new(database: &Arc<Box<dyn Database>>) -> Self {
Self {
database: database.clone(),
}
}
}

impl KeyRepository for DatabaseKeyRepository {
async fn get_key(&self, key: &Key) -> Result<Option<PeerKey>, Error> {
self.database.get_key_from_keys(key).map_err(Error::from)
}

async fn get_all(&self) -> Result<Vec<PeerKey>, Error> {
self.database.load_keys().map_err(Error::from)
```

```
}

async fn add_key(&self, peer_key: &PeerKey) -> Result<(), Error> {
self.database.add_key_to_keys(peer_key).map_err(Error::from)?;
Ok(())
}

async fn delete_key(&self, key: &Key) -> Result<(), Error> {
self.database.remove_key_from_keys(key).map_err(Error::from)?;
Ok(())
}
}
```

DatabaseDownloadsMetricRepository (`packages/tracker-core/src/statistics/persisted/downloads.rs`)

```
pub struct DatabaseDownloadsMetricRepository {
database: Arc<Box<dyn Database>>,
}

impl DatabaseDownloadsMetricRepository {
pub fn new(database: &Arc<Box<dyn Database>>) -> Self {
Self {
database: database.clone(),
```

```
}
}

pub fn load_all_torrents_downloads(&self) -> Result<NumberOfDownloadsBTreeMap,
self.database.load_all_torrents_downloads()
}

pub fn save_torrent_downloads(&self, info_hash: &InfoHash, downloads: NumberOf
self.database.save_torrent_downloads(info_hash, downloads)
}

pub fn increase_downloads_for_torrent(&self, info_hash: &InfoHash) -> Result<(
self.database.increase_downloads_for_torrent(info_hash)
}
}
```

7.2.2 데이터 접근 계층 설계

데이터 접근 계층은 다음과 같은 구조로 설계되어 있습니다:

1. **도메인 서비스**: 비즈니스 로직을 처리하는 상위 계층
2. **리포지토리**: 데이터 접근을 추상화하는 중간 계층
3. **데이터베이스 드라이버**: 실제 데이터베이스와 통신하는 하위 계층

7.3 R2D2 연결 풀 관리

7.3.1 연결 풀 아키텍처

Torrust Tracker는 R2D2 라이브러리를 사용하여 데이터베이스 연결 풀을 관리합니다:

SQLite 연결 풀 (`packages/tracker-core/src/databases/driver/sqlite.rs`)

```
pub(crate) struct Sqlite {
pool: Pool<SqliteConnectionManager>,
}

impl Sqlite {
pub fn new(db_path: &str) -> Result<Self, Error> {
let manager = SqliteConnectionManager::file(db_path);
let pool = r2d2::Pool::builder().build(manager).map_err(|e| (e, DRIVER))?;

Ok(Self { pool })
}
}
```

MySQL 연결 풀 (`packages/tracker-core/src/databases/driver/mysql.rs`)

```
pub(crate) struct Mysql {
pool: Pool<MySqlConnectionManager>,
}
```

```rust
impl Mysql {
pub fn new(db_path: &str) -> Result<Self, Error> {
let opts = Opts::from_url(db_path)?;
let builder = OptsBuilder::from_opts(opts);
let manager = MySqlConnectionManager::new(builder);
let pool = r2d2::Pool::builder().build(manager).map_err(|e| (e, DRIVER))?;

Ok(Self { pool })
}
}
```

7.3.2 연결 풀 사용 패턴

모든 데이터베이스 작업은 연결 풀에서 연결을 가져와 사용하는 패턴을 따릅니다:

```rust
// SQLite 예시
fn load_whitelist(&self) -> Result<Vec<InfoHash>, Error> {
let conn = self.pool.get().map_err(|e| (e, DRIVER))?;
let mut stmt = conn.prepare("SELECT info_hash FROM whitelist")?;
let info_hash_iter = stmt.query_map([], |row| {
let info_hash: String = row.get(0)?;
Ok(InfoHash::from_str(&info_hash).unwrap())
})?;
```

```
let info_hashes: Vec<InfoHash> = info_hash_iter.filter_map(std::result::Result:
Ok(info_hashes)
}


// MySQL 예시
fn load_whitelist(&self) -> Result<Vec<InfoHash>, Error> {
let mut conn = self.pool.get().map_err(|e| (e, DRIVER))?;
let info_hashes = conn.query_map("SELECT info_hash FROM whitelist", |info_hash:
InfoHash::from_str(&info_hash).unwrap()
})?;
Ok(info_hashes)
}
```

7.3.3 연결 풀 설정 및 최적화

R2D2 연결 풀은 기본 설정을 사용하지만, 필요에 따라 다음과 같은 설정을 조정할 수 있습니다:

- **최대 연결 수**: 동시에 유지할 수 있는 최대 연결 수
- **최소 유휴 연결 수**: 풀에서 유지할 최소 유휴 연결 수
- **연결 타임아웃**: 새 연결을 얻기 위한 최대 대기 시간
- **유휴 타임아웃**: 연결이 유휴 상태로 유지될 수 있는 최대 시간

7.4 SQLite vs MySQL 지원

7.4.1 드라이버 팩토리 패턴

두 데이터베이스 시스템을 지원하기 위해 팩토리 패턴을 사용합니다:

드라이버 팩토리 (`packages/tracker-core/src/databases/driver/mod.rs`)

```
#[derive(Serialize, Deserialize, PartialEq, Eq, Debug, derive_more::Display, Clo
pub enum Driver {
Sqlite3,
MySQL,
}

pub(crate) fn build(driver: &Driver, db_path: &str) -> Result<Box<dyn Database>
let database: Box<dyn Database> = match driver {
Driver::Sqlite3 => Box::new(Sqlite::new(db_path)?),
Driver::MySQL => Box::new(Mysql::new(db_path)?),
};

database.create_database_tables().expect("Could not create database tables.");
Ok(database)
}
```

7.4.2 데이터베이스별 구현 차이점

스키마 정의 차이점:

SQLite 스키마:

```sql
CREATE TABLE IF NOT EXISTS whitelist (
id INTEGER PRIMARY KEY AUTOINCREMENT,
info_hash TEXT NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS torrents (
id INTEGER PRIMARY KEY AUTOINCREMENT,
info_hash TEXT NOT NULL UNIQUE,
completed INTEGER DEFAULT 0 NOT NULL
);
```

MySQL 스키마:

```sql
CREATE TABLE IF NOT EXISTS whitelist (
id integer PRIMARY KEY AUTO_INCREMENT,
info_hash VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS torrents (
id integer PRIMARY KEY AUTO_INCREMENT,
info_hash VARCHAR(40) NOT NULL UNIQUE,
completed INTEGER DEFAULT 0 NOT NULL
);
```

UPSERT 구문 차이점:

SQLite:

```
conn.execute(
"INSERT INTO torrents (info_hash, completed) VALUES (?1, ?2) ON CONFLICT(info_ha
[info_hash.to_string(), completed.to_string()],
)?;
```

MySQL:

```
conn.exec_drop(
"INSERT INTO torrents (info_hash, completed) VALUES (:info_hash_str, :completed)
params! { info_hash_str, completed }
)?;
```

7.4.3 설정 기반 드라이버 선택

데이터베이스 설정 (`packages/configuration/src/v2_0_0/database.rs`)

```
#[derive(Serialize, Deserialize, PartialEq, Eq, Debug, Clone)]
pub struct Database {
#[serde(default = "Database::default_driver")]
pub driver: Driver,
#[serde(default = "Database::default_path")]
pub path: String,
```

```
}

#[derive(Serialize, Deserialize, PartialEq, Eq, PartialOrd, Ord, Debug, Hash, Cl
#[serde(rename_all = "lowercase")]
pub enum Driver {
Sqlite3,
MySQL,
}
```

초기화 과정 (`packages/tracker-core/src/databases/setup.rs`)

```
pub fn initialize_database(config: &Core) -> Arc<Box<dyn Database>> {
let driver = match config.database.driver {
torrust_tracker_configuration::Driver::Sqlite3 => Driver::Sqlite3,
torrust_tracker_configuration::Driver::MySQL => Driver::MySQL,
};

Arc::new(driver::build(&driver, &config.database.path).expect("Database driver b
}
```

7.5 데이터 마이그레이션 시스템

7.5.1 현재 마이그레이션 상태

현재 Torrust Tracker는 자동 마이그레이션을 지원하지 않습니다. 대신 애플리케이션 시작 시 하드코딩된 SQL 문을 사용하여 테이블을 생성합니다.

마이그레이션 README (`packages/tracker-core/migrations/README.md`)에 따르면:

> "We don't support automatic migrations yet. The tracker creates all the needed tables when it starts. The SQL sentences are hardcoded in each database driver."

7.5.2 수동 마이그레이션 파일

마이그레이션 파일들이 `packages/tracker-core/migrations/` 디렉토리에 준비되어 있습니다:

SQLite 마이그레이션:

- `20240730183000_torrust_tracker_create_all_tables.sql` : 초기 테이블 생성
- `20240730183500_torrust_tracker_keys_valid_until_nullable.sql` : 키 만료 시간 nullable 변경
- `20250527093000_torrust_tracker_new_torrent_aggregate_metrics_table.sql` : 집계 메트릭 테이블 추가

MySQL 마이그레이션:

- 동일한 파일명으로 MySQL 전용 SQL 구문 제공

7.5.3 스키마 버전 관리

현재는 명시적인 스키마 버전 관리 시스템이 없지만, 마이그레이션 파일의 타임스탬프 기반 명명 규칙을 사용합니다:

```
YYYYMMDDHHMMSS_description.sql
```

예시:

- `20240730183000` : 2024년 7월 30일 18:30:00
- `20240730183500` : 2024년 7월 30일 18:35:00

- `20250527093000` : 2025년 5월 27일 09:30:00

7.5.4 테이블 생성 로직

SQLite 테이블 생성 (`packages/tracker-core/src/databases/driver/sqlite.rs`)

```
fn create_database_tables(&self) -> Result<(), Error> {
let create_whitelist_table = "
CREATE TABLE IF NOT EXISTS whitelist (
id INTEGER PRIMARY KEY AUTOINCREMENT,
info_hash TEXT NOT NULL UNIQUE
);";

let create_torrents_table = "
CREATE TABLE IF NOT EXISTS torrents (
id INTEGER PRIMARY KEY AUTOINCREMENT,
info_hash TEXT NOT NULL UNIQUE,
completed INTEGER DEFAULT 0 NOT NULL
);";

let create_torrent_aggregate_metrics_table = "
CREATE TABLE IF NOT EXISTS torrent_aggregate_metrics (
id INTEGER PRIMARY KEY AUTOINCREMENT,
metric_name TEXT NOT NULL UNIQUE,
value INTEGER DEFAULT 0 NOT NULL
```

```
);";

let  create_keys_table = "
CREATE TABLE IF NOT EXISTS keys (
id INTEGER PRIMARY KEY AUTOINCREMENT,
key TEXT NOT NULL UNIQUE,
valid_until INTEGER
);";

let  conn = self.pool.get().map_err(|e| (e, DRIVER))?;

conn.execute(&create_whitelist_table, [])?;
conn.execute(&create_keys_table, [])?;
conn.execute(&create_torrents_table, [])?;
conn.execute(&create_torrent_aggregate_metrics_table, [])?;

Ok(())
}
```

7.6 트랜잭션 처리 및 데이터 일관성

7.6.1 트랜잭션 처리 방식

현재 구현에서는 대부분의 데이터베이스 작업이 단일 쿼리로 수행되어 명시적인 트랜잭션 관리가 필요하지 않습니다. 하지만 복잡한 작업의 경우 연결 풀에서 가져온 연결을 사용하여 트랜잭션을 관리할 수 있습니다.

SQLite 트랜잭션 예시:

```
fn complex_operation(&self) -> Result<(), Error> {
let mut conn = self.pool.get().map_err(|e| (e, DRIVER))?;
let tx = conn.transaction()?;
// 여러 작업 수행
tx.execute("INSERT INTO table1 ...", [])?;
tx.execute("UPDATE table2 ...", [])?;
tx.commit()?;
Ok(())
}
```

7.6.2 데이터 일관성 보장 메커니즘

UPSERT 패턴 사용:

데이터 일관성을 보장하기 위해 INSERT ... ON CONFLICT (SQLite) 또는 INSERT ... ON DUPLICATE KEY UPDATE (MySQL) 패턴을 사용합니다:

```
// SQLite
conn.execute(
"INSERT INTO torrents (info_hash, completed) VALUES (?1, ?2) ON CONFLICT(info_ha
[info_hash.to_string(), completed.to_string()],
)?;
```

```
// MySQL
conn.exec_drop(
"INSERT INTO torrents (info_hash, completed) VALUES (:info_hash_str, :completed)
params! { info_hash_str, completed }
)?;
```

원자적 업데이트:

카운터 증가와 같은 작업은 원자적으로 수행됩니다:

```
// 토렌트 다운로드 수 증가
conn.execute(
"UPDATE torrents SET completed = completed + 1 WHERE info_hash = ?",
[info_hash.to_string()],
)?;

// 전역 다운로드 수 증가
conn.execute(
"UPDATE torrent_aggregate_metrics SET value = value + 1 WHERE metric_name = ?",
[TORRENTS_DOWNLOADS_TOTAL],
)?;
```

7.6.3 에러 처리 및 롤백

에러 타입 정의 (`packages/tracker-core/src/databases/error.rs`)

```
#[derive(thiserror::Error, Debug, Clone)]

pub  enum Error {

#[error("The {driver} query unexpectedly returned nothing: {source}")]

QueryReturnedNoRows {

source: LocatedError<'static, dyn  std::error::Error + Send + Sync>,

driver: Driver,

},



#[error("The {driver} query was malformed: {source}")]

InvalidQuery {

source: LocatedError<'static, dyn  std::error::Error + Send + Sync>,

driver: Driver,

},



#[error("Unable to insert record into {driver} database, {location}")]

InsertFailed {

location: &'static Location<'static>,

driver: Driver,

},



#[error("Failed to create r2d2 {driver} connection pool: {source}")]
```

```
ConnectionPool {
source: LocatedError<'static, r2d2::Error>,
driver: Driver,
},
}
```

7.7 성능 최적화 및 인덱싱 전략

7.7.1 인덱스 설계

기본 인덱스:

모든 테이블에서 기본 키(Primary Key)가 자동으로 인덱싱됩니다:

```
-- SQLite
id INTEGER  PRIMARY KEY AUTOINCREMENT

-- MySQL
id integer  PRIMARY KEY AUTO_INCREMENT
```

고유 인덱스 (Unique Index):

중복을 방지하고 빠른 조회를 위해 고유 인덱스를 사용합니다:

```
-- info_hash 필드에 고유 인덱스
info_hash TEXT  NOT NULL  UNIQUE  -- SQLite
info_hash VARCHAR(40) NOT NULL  UNIQUE  -- MySQL
```

```
-- 인증 키에 고유 인덱스
key  TEXT  NOT NULL  UNIQUE  -- SQLite
`key`  VARCHAR(32) NOT NULL  UNIQUE  -- MySQL
```

7.7.2 쿼리 최적화 전략

인덱스 기반 조회:

모든 주요 조회 작업이 인덱싱된 필드를 사용합니다:

```
// info_hash로 토렌트 조회 (인덱싱된 필드)
"SELECT completed FROM torrents WHERE info_hash = ?"

// 키로 인증 정보 조회 (인덱싱된 필드)
"SELECT key, valid_until FROM keys WHERE key = ?"

// 화이트리스트에서 info_hash 조회 (인덱싱된 필드)
"SELECT info_hash FROM whitelist WHERE info_hash = ?"
```

배치 조회 최적화:

전체 데이터를 로드할 때는 단일 쿼리를 사용합니다:

```
// 모든 토렌트 메트릭을 한 번에 로드
"SELECT info_hash, completed FROM torrents"
```

```
// 모든 화이트리스트 항목을 한 번에 로드
"SELECT info_hash FROM whitelist"


// 모든 인증 키를 한 번에 로드
"SELECT key, valid_until FROM keys"
```

7.7.3 연결 풀 최적화

연결 재사용:

R2D2 연결 풀을 통해 연결을 재사용하여 연결 오버헤드를 최소화합니다:

```
// 연결 풀에서 연결 획득
let conn = self.pool.get().map_err(|e| (e, DRIVER))?;


// 연결 사용 후 자동으로 풀에 반환 (Drop trait 구현)
```

준비된 문장 (Prepared Statements):

SQL 인젝션 방지와 성능 향상을 위해 준비된 문장을 사용합니다:

```
// SQLite
let mut stmt = conn.prepare("SELECT info_hash FROM whitelist WHERE info_hash =
let mut rows = stmt.query([info_hash.to_hex_string()])?;


// MySQL
```

```
conn.exec_first::<String, _, _>(
"SELECT info_hash FROM whitelist WHERE info_hash = :info_hash",
params! { "info_hash" => info_hash.to_hex_string() },
)?;
```

7.7.4 메모리 사용 최적화

스트리밍 결과 처리:

대량의 데이터를 처리할 때 메모리 효율적인 스트리밍 방식을 사용합니다:

```
// SQLite - Iterator 기반 처리
let torrent_iter = stmt.query_map([], |row| {
let info_hash_string: String = row.get(0)?;
let info_hash = InfoHash::from_str(&info_hash_string).unwrap();
let completed: u32 = row.get(1)?;
Ok((info_hash, completed))
})?;

// 결과를 필터링하면서 수집
Ok(torrent_iter.filter_map(std::result::Result::ok).collect())
```

효율적인 데이터 구조:

조회 성능을 위해 적절한 데이터 구조를 사용합니다:

```
// BTreeMap을 사용하여 정렬된 키-값 저장
```

```
pub type NumberOfDownloadsBTreeMap = BTreeMap<InfoHash, NumberOfDownloads>;
```

7.8 데이터베이스 테스트 전략

7.8.1 통합 테스트 구조

공통 테스트 프레임워크 (`packages/tracker-core/src/databases/driver/mod.rs`)

```
pub async fn run_tests(driver: &Arc<Box<dyn Database>>) {
database_setup(driver).await;

// 토렌트 영속성 테스트
handling_torrent_persistence::it_should_save_and_load_persistent_torrents(driver
handling_torrent_persistence::it_should_load_all_persistent_torrents(driver);
handling_torrent_persistence::it_should_increase_the_number_of_downloads_for_a_g

// 인증 키 테스트
handling_authentication_keys::it_should_load_the_keys(driver);
handling_authentication_keys::it_should_save_and_load_permanent_authentication_k
handling_authentication_keys::it_should_save_and_load_expiring_authentication_ke

// 화이트리스트 테스트
handling_the_whitelist::it_should_load_the_whitelist(driver);
handling_the_whitelist::it_should_add_and_get_infohashes(driver);
```

```
handling_the_whitelist::it_should_remove_an_infohash_from_the_whitelist(driver);
}
```

7.8.2 드라이버별 테스트 설정

SQLite 테스트:

```
fn ephemeral_configuration() -> Core {
let mut config = Core::default();
let temp_file = ephemeral_sqlite_database();
temp_file.to_str().unwrap().clone_into(&mut config.database.path);
config
}

#[tokio::test]
async fn run_sqlite_driver_tests() -> Result<(), Box<dyn std::error::Error +
let config = ephemeral_configuration();
let driver = initialize_driver(&config);
run_tests(&driver).await;
Ok(())
}
```

MySQL 테스트 (Testcontainers 사용):

```
#[tokio::test]

async fn run_mysql_driver_tests() -> Result<(), Box<dyn std::error::Error + '

if std::env::var("TORRUST_TRACKER_CORE_RUN_MYSQL_DRIVER_TEST").is_err() {

println!("Skipping the MySQL driver tests.");

return Ok(());

}



let mysql_configuration = MysqlConfiguration::default();

let stopped_mysql_container = StoppedMysqlContainer::default();

let mysql_container = stopped_mysql_container.run(&mysql_configuration).await.u



let host = mysql_container.get_host().await;

let port = mysql_container.get_host_port_ipv4().await;

let config = core_configuration(&host, port, &mysql_configuration);

let driver = initialize_driver(&config);



run_tests(&driver).await;

mysql_container.stop().await;

Ok(())

}
```

7.9 보안 고려사항

7.9.1 SQL 인젝션 방지

모든 데이터베이스 쿼리에서 준비된 문장(Prepared Statements)을 사용하여 SQL 인젝션을 방지합니다:

```
// 안전한 파라미터 바인딩 - SQLite
conn.execute(
"INSERT INTO whitelist (info_hash) VALUES (?)",
[info_hash.to_string()]
)?;

// 안전한 파라미터 바인딩 - MySQL
conn.exec_drop(
"INSERT INTO whitelist (info_hash) VALUES (:info_hash_str)",
params! { info_hash_str },
)?;
```

7.9.2 연결 문자열 보안

비밀번호 마스킹 (`packages/configuration/src/v2_0_0/database.rs`)

```
pub fn mask_secrets(&mut self) {
match self.driver {
Driver::Sqlite3 => {
// SQLite는 마스킹할 비밀번호가 없음
}
```

```
Driver::MySQL => {
let mut url = Url::parse(&self.path).expect("path for MySQL driver should be a
url.set_password(Some("***")).expect("url password should be changed");
self.path = url.to_string();
}
}
}
```

7.9.3 에러 정보 노출 제한

데이터베이스 에러를 적절히 추상화하여 내부 구현 세부사항이 노출되지 않도록 합니다:

```
impl From<r2d2_sqlite::rusqlite::Error> for Error {
#[track_caller]
fn from(err: r2d2_sqlite::rusqlite::Error) -> Self {
match err {
r2d2_sqlite::rusqlite::Error::QueryReturnedNoRows => Error::QueryReturnedNoRows
source: (Arc::new(err) as DynError).into(),
driver: Driver::Sqlite3,
},
_ => Error::InvalidQuery {
source: (Arc::new(err) as DynError).into(),
driver: Driver::Sqlite3,
```

```
},
}
}
}
```

7.10 결론

Torrust Tracker의 데이터베이스 연동 시스템은 다음과 같은 강점을 가지고 있습니다:

7.10.1 아키텍처적 장점

1. **추상화**: Database 트레이트를 통한 깔끔한 추상화
2. **확장성**: 새로운 데이터베이스 드라이버 추가 용이
3. **테스트 가능성**: Mock 지원으로 단위 테스트 용이
4. **분리**: 도메인 로직과 데이터 접근 로직의 명확한 분리

7.10.2 성능 및 안정성

1. **연결 풀링**: R2D2를 통한 효율적인 연결 관리
2. **인덱싱**: 적절한 인덱스 설계로 빠른 조회 성능
3. **에러 처리**: 포괄적인 에러 처리 및 복구 메커니즘
4. **트랜잭션**: 데이터 일관성 보장을 위한 원자적 연산

7.10.3 개선 가능한 영역

1. **자동 마이그레이션**: 현재 수동 마이그레이션에서 자동화된 시스템으로 발전 필요
2. **연결 풀 튜닝**: 워크로드에 따른 연결 풀 설정 최적화

3. **모니터링**: 데이터베이스 성능 메트릭 수집 및 모니터링 강화

4. **백업 및 복구**: 자동화된 백업 및 복구 시스템 구축

이러한 데이터베이스 연동 아키텍처는 Torrust Tracker가 높은 성능과 안정성을 유지하면서도 확장 가능한 영속성 계층을 제공할 수 있게 해줍니다.